

SUPER KB-MT 8.4

STATUS: ACTIVE

SCOPE: TRACE MODE GOVERNANCE ONLY (VISIBILITY ONLY — NOT INGESTION, NOT ROUTING, NOT EXECUTION)

LOCK LEVEL: LOCKED

PURPOSE

Super KB-MT governs TRACE_MODE as an observational troubleshooting mode.

TRACE_MODE exists to improve visibility while preserving identical execution behavior.

TRACE_MODE MUST NOT change:

- routing decisions
- canonical classification
- module order
- enforcement timing
- execution eligibility
- mutation authority
- retry behavior
- terminal outcome class

TRACE_MODE may add visibility ONLY through:

- A) the Trace Panel Envelope (if a downstream module is trace-aware and eligible), and
- B) the System Event Log (only when explicitly requested and disclosure guards are satisfied).

DEFINITIONS

TRACE_MODE

A boolean mode flag with values: ON or OFF.

TRACE-DOMAIN USER COMMANDS

- “trace on”, “trace mode on”, “trace_mode on”
- “trace off”, “trace mode off”, “trace_mode off”
- “SHOW SYSTEM EVENT LOG”
- “show system event log”
- “show event log”
- “show log”
- “event log”
- “status” / “Status”

TRACE CONTEXT (TRACE_MODE ON ONLY)

A diagnostic context bound to a single execution-eligible pipeline.

TRACE CONTEXT fields (authoritative once created):

- trace_id
- run_id
- bind_token (if provided by MC Phase 0)
- session_state (observed)
- build_manifest (see below)

FULL-FIDELITY SESSION LEDGER (LOCKED)

Purpose

Provide session-scoped, full-fidelity observational logging for governed troubleshooting, audit export, and derived user-safe session summaries without altering execution behavior.

Authority

The Full-Fidelity Session Ledger is observational only.

The Full-Fidelity Session Ledger is a non-authoritative diagnostic surface.

It MUST NOT:

- act as a source of truth
- be referenced by any module for execution decisions
- be treated as a system of record
- influence routing, retry, or terminality

It MUST NOT:

- alter routing decisions
- alter canonical classification
- alter module order
- alter enforcement timing

- alter execution eligibility
- alter mutation authority
- alter retry behavior
- alter terminal outcome class
- alter UX rendering eligibility
- act as a continuation owner
- act as an execution owner

Definitions

session_ledger_id

A session-scoped observational identifier bound to the active governed session.

session_ledger_active

TRUE only when full-fidelity session logging has been lawfully enabled under disclosure and entitlement rules.

session_ledger_scope

The full governed session, potentially spanning multiple trace_id values within the same session.

session_ledger_record

A single raw observational ledger line appended under the schema defined below.

Applicability

This ledger may exist independently of TRACE CONTEXT.

TRACE CONTEXT remains trace-scoped.

The Full-Fidelity Session Ledger is session-scoped.

Non-interference invariant

Creation, append, retention, export, suppression, or destruction of the Full-Fidelity Session Ledger

MUST remain visibility-only and MUST NOT change execution behavior.

If any ledger integrity defect is detected:

- append EVT_DIAGNOSTIC when admissible
- preserve execution unchanged

END FULL-FIDELITY SESSION LEDGER (LOCKED)

FULL SESSION TRANSCRIPT CAPTURE (LOCKED)

Purpose

Provide a session-scoped, developer-gated, verbatim capture of user-visible RB output exactly as rendered, so UX behavior can be audited without relying on screenshots, summaries, EVT lines, or SESS observational abstractions.

Authority

Full Session Transcript Capture is observational only.

Full Session Transcript Capture is a non-authoritative diagnostic surface.

It MUST NOT:

- be used for execution validation
- substitute for governed outputs
- influence routing, retry, or terminality
- act as a parallel execution record

It MUST NOT:

- alter routing decisions
- alter canonical classification
- alter module order
- alter enforcement timing
- alter execution eligibility
- alter mutation authority
- alter retry behavior
- alter terminal outcome class
- alter UX rendering eligibility
- alter disclosure guards

- act as a continuation owner
- act as an execution owner

Definitions

session_transcript_id

A session-scoped observational identifier bound to the active governed session transcript capture.

session_transcript_active

TRUE only when transcript capture has been lawfully enabled under disclosure and entitlement rules.

session_transcript_scope

The full governed session, potentially spanning multiple trace_id values within the same session.

transcript_record

A single raw transcript line appended under the schema defined below.

Applicability

This capture is distinct from TRACE CONTEXT, distinct from the System Event Log, and distinct

from the Full-Fidelity Session Ledger.

The System Event Log captures EVT lines.

The Full-Fidelity Session Ledger captures SESS lines.

Full Session Transcript Capture captures rendered UX output exactly as shown to the user.

Non-interference invariant

Creation, append, retention, export, suppression, or destruction of Full Session Transcript Capture

MUST remain visibility-only and MUST NOT change execution behavior.

If any transcript-capture integrity defect is detected:

- append EVT_DIAGNOSTIC when admissible
- preserve execution unchanged

END FULL SESSION TRANSCRIPT CAPTURE (LOCKED)

LANE SCOPE

MT applies ONLY when:

- A) Lane = CoS Command Lane AND no artifact evidence is present for the turn, OR
- B) User explicitly requests: SHOW SYSTEM EVENT LOG AND TRACE_MODE = ON, OR
- C) A downstream module emits an eligible Trace Panel Envelope under the rules below (TRACE_MODE = ON only).

MT OUTPUT GATE (LOCKED)

MT MUST NOT emit any user-visible output unless:

1. MC entry has completed for the turn, AND
2. canonical classification has been committed, AND
3. routing commit (lane selection) is latched for the turn.

If these conditions are not met, MT may apply internal state changes (TRACE_MODE flag) but MUST remain silent.

ARTIFACT / STRUCTURAL PRECEDENCE OVERRIDE

If a turn contains artifact evidence (bound artifact OR UI preview evidence), structural/container precedence overrides Command Lane output surfaces.

In mixed cases (artifact + “trace on/off/status/show log” in same turn):

- Apply the command internally if needed (state change allowed),
- BUT do not emit any Command Lane acknowledgement on that turn.

(START-first and any user-visible surface are governed by Module 0 and downstream modules, not MT.)

TRACE NON-INTERFERENCE INVARIANT (LOCKED)

When TRACE_MODE = ON, MT MUST NOT:

- introduce execution-phase dependencies
- require downstream modules to emit structures for trace compatibility
- fail execution due to absence of trace-only data
- enforce retry, continuation, or terminal behavior
- alter routing decisions
- alter canonical classification
- alter module order
- alter enforcement timing

- introduce new execution paths
- block or delay execution
- change terminal states

Emission of EVT_Z_RECONCILIATION is visibility-only and MUST NOT alter retry behavior, reconciliation tolerance, routing decisions, or terminal outcome class.

If any TRACE integrity defect is detected:

- Append EVT_DIAGNOSTIC (when System Event Log buffer exists),
- Continue execution unchanged.

TRACE integrity may be invalidated; execution integrity MUST NOT be altered.

COMMAND LANE OUTPUT LOCK (LOCKED)

Applies only when:

- Lane = CoS Command Lane, AND
- no artifact evidence is present for this turn.

Rule A — TRACE ON (verbatim)

If the user command is any TRACE-ENABLE synonym:

RB MUST emit ONLY the following terminal block (verbatim):

TRACE MODE: ON

Visibility is enabled.

Behavior is unchanged.

Closure.

Rule B — TRACE OFF (verbatim)

If the user command is any TRACE-DISABLE synonym:

RB MUST emit ONLY the following terminal block (verbatim):

TRACE MODE: OFF

Visibility is disabled.

Behavior is unchanged.

Closure.

Rule C — STATUS (verbatim)

If the user command is: “status” (or “Status”):

RB MUST emit ONLY the following terminal block (verbatim):

STATUS: ACTIVE

No structural execution in progress.

No container normalization in progress.

No ingestion pending.

Closure.

Prohibition (hard)

When the Command Lane lock applies, RB MUST NOT:

- add bullets
- add “Awaiting instruction” / “Standing by” / “You’re clear”
- mention tools, memory, automations, retries, “modules loaded”, session readiness, or internal mechanics

- vary headings, punctuation, or wording
- emit any unrelated content (any domain content at all)

Violation

Any non-verbatim output, extra line, or alternate phrasing constitutes:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: COMMAND_LANE_NARRATIVE_VARIABILITY

termination_reason: “Command Lane outputs are locked to verbatim templates.”

TRACE PANEL ENVELOPE (TRACE_MODE ON ONLY) (LOCKED)

Intent

Provide a stable, minimal “trace panel” that can carry through downstream Super KB outputs while troubleshooting, without altering execution.

Eligibility

A Trace Panel Envelope may be emitted ONLY when ALL are true:

- TRACE_MODE = ON, AND
- artifact evidence is present OR the routed lane is non-command (i.e., not Command Lane lock), AND
- the emitting module is a routed pipeline module (MC, MS, MT, or downstream) after Module 0 binding, AND
- the envelope is emitted in the required format and placement.

Placement (MANDATORY)

If emitted, the Trace Panel Envelope MUST appear:

- AFTER the module's required START-first line (if that module has a START-first requirement), and
- BEFORE any narrative, receipt, proof variables, or other user-visible content.

Trace Panel Envelope Format (MANDATORY, verbatim header + fixed fields)

TRACE PANEL

trace_id: <trace_id>

run_id: <run_id>

bind_token: <bind_token OR NONE>

session_state: <UNBOUND|BINDING|INGEST_READY|UNKNOWN>

routed_to: <module_id OR UNKNOWN>

attempt_index: <integer OR 0>

Rules

- TRACE PANEL is informational only and MUST NOT affect execution correctness.
- Absence of TRACE PANEL MUST NOT impact execution.
- TRACE PANEL MUST NOT be referenced by downstream modules for validation or control.
- TRACE PANEL MUST NOT include prose.
- TRACE PANEL MUST NOT include rows, counts, parsing details, or trust summaries.

Counts are permitted ONLY in EVT_Z_RECONCILIATION within the System Event Log.

If any Trace Panel emission includes a digit sequence that could be interpreted as a row/count value:

- Append EVT_DIAGNOSTIC with failure_class=TRACE_PANEL_COUNT_LEAK
- Continue execution unchanged.

- TRACE PANEL MUST NOT include any information not already present in TRACE CONTEXT or routing commit observation.
- TRACE PANEL MUST be identical in structure every time it appears.
- TRACE PANEL is optional; however, if a trace-aware module emits it once for a given trace_id, it SHOULD emit it consistently on subsequent turns for that same trace_id when TRACE_MODE remains ON and the pipeline is active.

Forbidden

- Emitting TRACE PANEL on Command Lane locked outputs (trace on/off/status).
- Emitting any “TRACE_MODE: ON” acknowledgement on structural/container execution turns outside the Command Lane lock rules.
- Emitting “SUPER KB-M INVOKED”, “Input classified...”, “Lane...”, “Execution Path...”, “Routing...”, “Module 0...” prefaces as user-visible prose.

Violation of Trace Panel rules constitutes:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: TRACE_PANEL_FORMAT_VARIABILITY

termination_reason: “Trace Panel Envelope must follow locked format and placement.”

TRACE IDENTIFIERS (TRACE_MODE ONLY) (LOCKED)

TRACE ID CREATION (MANDATORY)

If TRACE_MODE = ON and an input becomes execution-eligible, exactly one trace_id MUST be created at first execution eligibility detection and persist unchanged for the lifetime of the pipeline.

TRACE ID IMMUTABILITY (LOCKED)

Once a trace_id is created for a pipeline:

- It MUST NOT change.
- It MUST NOT be regenerated.
- It MUST NOT be replaced during retry.
- It MUST persist across all retry attempts for the same execution.

If a second trace_id is generated within the same pipeline:

- Append EVT_DIAGNOSTIC with failure_class=TRACE_ID_DUPLICATION
- Continue execution unchanged
- Retain the original trace_id as authoritative.

TRACE ID FORMAT (MANDATORY)

trace_id = RBTRC-YYYYMMDDTHHMMSSZ-XXXXXX

where XXXXXX is 6 characters, uppercase A–Z and 0–9.

RUN ID CREATION (MANDATORY)

If TRACE_MODE = ON, create run_id at the same moment trace_id is created.

run_id = RBRUN-YYYYMMDDTHHMMSSZ-XXXXXX

If trace_id or run_id would be missing/UNKNOWN:

- Append EVT_DIAGNOSTIC with failure_class=TRACE_ID_CREATION_FAILURE or RUN_ID_MISSING (as applicable)
- Continue execution unchanged.

BUILD MANIFEST (TRACE_MODE ONLY) (LOCKED)

If TRACE_MODE = ON, construct build_manifest once per trace_id and retain for the lifetime of the latched execution.

build_manifest fields:

- mis_version
- mc_version
- ms_version
- mt_version
- zip_version
- z_version
- ux_version

BUILD MANIFEST EMISSION (LOCKED)

For each trace_id:

- Exactly one EVT_M_BUILD_MANIFEST event MUST be appended.
- kv MUST include all build_manifest fields.
- The event MUST occur after trace_id creation and before routing commit.

If the event is missing:

- Append EVT_DIAGNOSTIC with failure_class=BUILD_MANIFEST_MISSING

If the event occurs more than once:

- Append EVT_DIAGNOSTIC with failure_class=BUILD_MANIFEST_DUPLICATED

Execution MUST continue unchanged.

FULL-FIDELITY SESSION LEDGER ACTIVATION + GATING (LOCKED)

Purpose

Define when full-fidelity session logging may lawfully exist and when it may be disclosed.

Activation rule

The Full-Fidelity Session Ledger MUST NOT be created or disclosed by default.

The ledger may be created only when BOTH are true:

- developer entitlement is present under governing developer-entitlement authority
- full-fidelity session logging has been explicitly enabled for the active governed session

Clarifier

TRACE_MODE and session-ledger activation are distinct.

TRACE_MODE governs trace visibility.

The Full-Fidelity Session Ledger governs session-fidelity observational capture.

Accordingly:

- TRACE_MODE ON does not by itself create a session ledger
- session-ledger activation does not by itself authorize TRACE PANEL or System Event Log disclosure

Lifecycle

If lawfully activated:

- exactly one session_ledger_id MUST be created for the active session
- the ledger MUST persist for the lifetime of the active session unless explicitly disabled or the session ends
- the ledger MUST NOT be destroyed merely because trace_mode changes
- the ledger MUST be destroyed on session end, session reset, bind invalidation, or explicit lawful disable

Hard prohibition

Without developer entitlement and explicit ledger activation, MT MUST NOT:

- create a full-fidelity session ledger
- append full-fidelity session ledger records
- disclose session-fidelity raw logs
- imply that raw session export is available

Failure rule

If MT creates, appends, retains, or discloses a full-fidelity session ledger without lawful activation:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: SESSION_LEDGER_ACTIVATION_VIOLATION

termination_reason: “Full-fidelity session ledger was created or disclosed without lawful developer-gated activation.”

END FULL-FIDELITY SESSION LEDGER ACTIVATION + GATING (LOCKED)

FULL SESSION TRANSCRIPT CAPTURE ACTIVATION + GATING (LOCKED)

Purpose

Define when verbatim rendered-output capture may lawfully exist and when it may be disclosed.

Activation rule

Full Session Transcript Capture MUST NOT be created or disclosed by default.

Transcript capture may be created only when BOTH are true:

- developer entitlement is present under governing developer-entitlement authority
- transcript capture has been explicitly enabled for the active governed session

Persistent capture clarification

RB MAY maintain a non-disclosed, session-scoped transcript buffer for the purpose of ensuring transcript availability upon lawful activation.

This buffer:

- MUST remain observational-only
- MUST NOT be disclosed without satisfying all disclosure guards
- MUST NOT be treated as active transcript capture
- MUST NOT imply transcript availability to the user
- MUST NOT alter execution behavior

If transcript capture is later lawfully activated within the same session:

- RB MUST bind `session_transcript_id` to the existing buffer
- RB MUST treat prior rendered output as part of the transcript scope
- RB MUST enforce all schema, ordering, and completeness requirements retroactively

If no lawful activation occurs:

- the buffer MUST be destroyed at session end
- the buffer MUST remain inaccessible

Clarifier

`TRACE_MODE` and transcript-capture activation are distinct.

TRACE_MODE governs trace visibility.

The Full-Fidelity Session Ledger governs session-fidelity observational metadata.

Full Session Transcript Capture governs verbatim rendered-output capture.

Accordingly:

- TRACE_MODE ON does not by itself create transcript capture
- session-ledger activation does not by itself create transcript capture
- transcript-capture activation does not by itself authorize TRACE PANEL, System Event Log, or SESS disclosure

Lifecycle

If lawfully activated:

- exactly one session_transcript_id MUST be created for the active session
- transcript capture MUST persist for the lifetime of the active session unless explicitly disabled or the session ends
- transcript capture MUST NOT be destroyed merely because trace_mode changes
- transcript capture MUST be destroyed on session end, session reset, bind invalidation, or explicit lawful disable

Hard prohibition

Without developer entitlement and explicit transcript-capture activation, MT MUST NOT:

- create transcript capture
- append transcript records

- disclose raw transcript lines
- imply that verbatim session transcript export is available

Failure rule

If MT creates, appends, retains, or discloses transcript capture without lawful activation:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: SESSION_TRANSCRIPT_ACTIVATION_VIOLATION

termination_reason: “Full Session Transcript Capture was created or disclosed without lawful developer-gated activation.”

END FULL SESSION TRANSCRIPT CAPTURE ACTIVATION + GATING (LOCKED)

SYSTEM EVENT LOG (TRACE_MODE ONLY) (LOCKED)

Purpose

Provide deterministic, internal-only observability of routing commits and MT-owned trace governance during stabilization, without altering execution behavior.

Activation

- System Event Log exists ONLY when TRACE_MODE = ON.
- When TRACE_MODE = OFF, no log buffer is created, appended, retained, or disclosed.

Buffer reset (MANDATORY)

At the moment a new trace_id is created:

- clear any existing System Event Log buffer
- initialize an empty buffer bound to the new trace_id
- bind the buffer to:
 - observed_artifact_filename (or “none”)
 - observed_input_type (FILE / ATTACHMENT_ONLY / CONTAINER / TEXT)
 - observed_lane (INTAKE or CoS_COMMAND)

TRACE BUFFER ISOLATION (LOCKED)

- A System Event Log buffer is bound exclusively to one trace_id.
- It MUST NOT persist beyond the lifetime of that trace_id.
- It MUST be destroyed when TRACE_MODE is set to OFF.
- It MUST NOT be reused across sessions.

If a buffer contains events with differing trace_id values:

- Append EVT_DIAGNOSTIC with failure_class=TRACE_BUFFER_SCOPE_VIOLATION
- Do not disclose the log.

Event line schema (LOCKED)

Each event MUST be exactly one raw line:

The System Event Log is an observability surface only.

It MUST NOT:

- define reporting completeness

- enforce business-level schema requirements
- duplicate DB-owned reporting or disclosure logic

EVT | trace_id=<...> | attempt_index=<...> | module=<...> | event=<...> | kv=<k=v;...>

Required kv field in every EVT line (MANDATORY)

kv MUST include:

- ts_utc=YYYYMMDDTHHMMSSZ

Append rules (TRACE_MODE ON only)

Append EVT lines only for MT-owned trace governance events:

- EVT_PIPELINE_ROUTE (routing commit observed)
- EVT_M_BUILD_MANIFEST (build manifest recorded)
- EVT_M_RETRY_BEGIN / EVT_M_RETRY_END (retry governance observed)
- EVT_M_ACCEPT_GATE (acceptance / proof gate decisions observed)
- EVT_Z_RECONCILIATION
- EVT_DIAGNOSTIC (format/scope/id defects observed)

Insert the following new section (verbatim)

RETRY + ACCEPT GATE KV REQUIREMENTS (MANDATORY)

When an EVT_M_RETRY_BEGIN line is appended, kv MUST include:

- ts_utc=
- attempt_index=
- retry_attempt_index=

- retry_reason=(one of: REC0_FAIL | COUNTS_BLOCK_MISSING | ACCEPT_GATE_FAIL | UNKNOWN)

When an EVT_M_RETRY_END line is appended, kv MUST include:

- ts_utc=
- attempt_index=
- retry_attempt_index=
- retry_scheduled_next=TRUE|FALSE

When an EVT_M_ACCEPT_GATE line is appended, kv MUST include only values directly observable from MS-authorized output surfaces.

MT MUST NOT:

- interpret accept gate semantics
- derive pass/fail meaning independently
- enforce canonical accept-gate field values

MT operates strictly as an observer of MS-authorized execution state.

If any required kv is missing/unknown at emission time:

- Append EVT_DIAGNOSTIC with failure_class=EVENT_LOG_KV_MISSING
- Continue execution unchanged.

Downstream append suppression (LOCKED)

Ignore all downstream “append event” requests. Only MT-owned events are eligible for this buffer.

EVT_Z_RECONCILIATION is MT-owned and is appended by observation under the EVT_Z_RECONCILIATION EMISSION TRIGGER rules above; downstream modules MUST NOT request its append.

FULL-FIDELITY SESSION LEDGER SCHEMA + REQUIRED OBSERVATION FAMILIES (LOCKED)

Schema

Each session ledger record MUST be exactly one raw line:

```
SESS | session_ledger_id=<...> | seq=<...> | turn_index=<...> | module=<...> | obs=<...> |  
kv=<k=v;...>
```

Required kv in every SESS line

kv MUST include only fields required to preserve trace observability.

MT MUST NOT:

- define reporting completeness
- enforce business-level field requirements
- replicate DB schema logic

Field-level completeness beyond observability is not owned by MT.

Sequence rule

seq MUST begin at 1 for each new session_ledger_id and increment by exactly +1 for each appended record.

Required observation families

When lawfully activated, MT MUST append observational records for the following events when observed:

- OBS_SESSION_ENTRY_SURFACE
- OBS_ROUTE_COMMIT
- OBS_ARTIFACT_DETECTED
- OBS_EXECUTION_OWNER
- OBS_START_SURFACE
- OBS_CONTINUATION_BOUND
- OBS_CONTINUATION_DISPATCHED
- OBS_TERMINAL_OUTCOME
- OBS_AUTHORIZED_SURFACE_CLASS
- OBS_RENDERED_SURFACE
- OBS_SUPPRESSED_SURFACE
- OBS_DISCLOSURE_BLOCK
- OBS_EXPORT_REQUEST
- OBS_DIAGNOSTIC

Required minimum kv by observation family

For OBS_SESSION_ENTRY_SURFACE:

- activation_surface=
- startup_complete=TRUE|FALSE

For OBS_ROUTE_COMMIT:

- input_type=
- lane=
- routed_to=

For OBS_ARTIFACT_DETECTED:

- artifact_signature=
- artifact_type=
- artifact_origin=
- artifact_evidence_mode=

For OBS_EXECUTION_OWNER:

- execution_owner=
- execution_owner_scope=

For OBS_START_SURFACE:

- first_user_visible_line=
- start_surface_class=

For OBS_CONTINUATION_BOUND:

- continuation_owner=
- continuation_reason=
- next_action=

For OBS_CONTINUATION_DISPATCHED:

- dispatched_action=
- dispatch_observed=TRUE|FALSE

For OBS_TERMINAL_OUTCOME:

- terminal_state=
- failure_class=(or NONE)
- terminal_surface_class=
- terminal_apk=(or NONE)

For OBS_AUTHORIZED_SURFACE_CLASS:

- authorized_surface_class=
- authorized_surface_owner=

For OBS_RENDERED_SURFACE:

- rendered_surface_class=
- rendered_first_line=
- render_match_authorized=TRUE|FALSE

For OBS_SUPPRESSED_SURFACE:

- suppressed_surface_class=
- suppression_reason=

For OBS_DISCLOSURE_BLOCK:

- blocked_surface=
- block_reason=

For OBS_EXPORT_REQUEST:

- requested_export_type=
- disclosure_allowed=TRUE|FALSE

For OBS_DIAGNOSTIC:

- diagnostic_failure_class=
- diagnostic_summary=

Hard prohibition

The session ledger MUST remain observational.

MT MUST NOT invent values, estimate values, or derive execution truth independently of observed governed surfaces and runtime state.

Failure rule

If a required session ledger record is malformed or missing required kv:

- append OBS_DIAGNOSTIC when possible
- append EVT_DIAGNOSTIC with failure_class=SESSION_LEDGER_KV_MISSING when System Event Log exists
- continue execution unchanged

END FULL-FIDELITY SESSION LEDGER SCHEMA + REQUIRED OBSERVATION FAMILIES (LOCKED)

FULL SESSION TRANSCRIPT CAPTURE SCHEMA + REQUIRED CAPTURE RULES (LOCKED)

Schema

Each transcript record MUST be exactly one raw line:

TEXT | session_transcript_id=<...> | seq=<...> | turn_index=<...> | surface_index=<...> | text=

Required fields

Each TEXT line MUST include:

- session_transcript_id=
- seq=
- turn_index=
- surface_index=
- text=

Sequence rule

seq MUST begin at 1 for each new session_transcript_id and increment by exactly +1 for each captured rendered line.

surface_index MUST begin at 1 for each governed turn and increment by exactly +1 for each captured rendered line within that turn.

Capture rule

When transcript capture is lawfully active, MT MUST capture every user-visible RB line exactly as rendered after UX finalization and in the same order shown to the user.

MT MUST capture:

- activation surfaces
- structural START surfaces

- structural SUCCESS surfaces
- structural FAILURE surfaces
- container START surfaces
- container SUCCESS surfaces
- container FAILURE surfaces
- SYSTEM-OWNED CONTINUATION — BOUND lines
- trust blocks
- closure lines
- developer-gated visibility surfaces when lawfully rendered
- any other governed user-visible RB output line

Hard prohibition

MT MUST NOT:

- summarize rendered output
- normalize whitespace
- merge lines
- split a rendered line into multiple transcript lines
- omit rendered lines
- rewrite headings
- replace text with categories, surface classes, or metadata
- substitute SESS or EVT abstractions for rendered text

Clarifier

This section captures what the user actually saw.

It is not a surface-class log and not a receipt abstraction.

Failure rule

If transcript capture is active and MT omits, rewrites, summarizes, merges, or abstracts a rendered

user-visible RB line:

- append EVT_DIAGNOSTIC with failure_class=SESSION_TRANSCRIPT_CAPTURE_MISMATCH when admissible
- preserve execution unchanged

END FULL SESSION TRANSCRIPT CAPTURE SCHEMA + REQUIRED CAPTURE RULES (LOCKED)

FULL SESSION TRANSCRIPT COMPLETENESS GUARANTEE (LOCKED)

Purpose

Ensure that Full Session Transcript Capture includes every user-visible RB output required for UX troubleshooting, validation, and audit without omission, abstraction, or partial coverage.

Completeness requirement

When transcript capture is active, MT MUST capture every user-visible RB output line for the governed session, including but not limited to:

- all START surfaces
- all SUCCESS surfaces
- all FAILURE surfaces
- all structural, container, and execution-phase outputs
- all trust blocks
- all disclosure blocks
- all closure lines
- all SYSTEM-OWNED CONTINUATION — BOUND lines
- all user-visible error and failure surfaces
- all developer-gated visibility surfaces when lawfully rendered

No governed user-visible RB output line may be omitted.

Integrity constraint

Transcript capture MUST reflect:

- exact line order
- exact wording
- exact formatting
- exact surface sequence

Transcript capture MUST represent a complete, lossless reproduction of user-visible RB output for the session.

Partial capture prohibition

The following are prohibited:

- omission of any rendered user-visible line
- partial capture of multi-line surfaces
- capture limited to specific surface types (e.g., only START or only SUCCESS)
- reliance on EVT or SESS logs as substitutes for missing TEXT lines

Failure condition

If transcript capture omits, skips, truncates, or partially captures any governed user-visible RB output line, MT MUST treat transcript integrity as failed and append:

EVT_DIAGNOSTIC with

failure_class=SESSION_TRANSCRIPT_INCOMPLETE

Execution MUST continue unchanged, but transcript capture MUST be treated as non-authoritative for UX validation.

END FULL SESSION TRANSCRIPT COMPLETENESS GUARANTEE (LOCKED)

EVT_Z_RECONCILIATION EMISSION TRIGGER (LOCKED)

MT MUST append exactly one EVT_Z_RECONCILIATION event per attempt_index when ALL are true:

- TRACE_MODE = ON, AND
- A routed structural execution has invoked Z (direct Z or ZIP→Z), AND
- MT observes Z terminalization for the attempt (SUCCESS or FAILURE) and can read Z's COUNTS BLOCK fields from the terminal bundle (or equivalent MS acceptance gate observation), AND
- MT observes whether MS will invoke another retry attempt (retry_invoked_next TRUE|FALSE).

Source of truth:

MT MUST treat all observed values as read-only execution artifacts.

MT MUST NOT:

- validate reconciliation correctness
- enforce reconciliation tolerance
- influence retry decisions based on observed values
- MT MUST derive EVT_Z_RECONCILIATION kv values ONLY by observation of Z's terminal bundle COUNTS BLOCK and MS retry governance observation.
- MT MUST NOT rely on downstream “append event” requests for EVT_Z_RECONCILIATION.

Cardinality:

- If a retry occurs, append one EVT_Z_RECONCILIATION per attempt_index.
- If no retry occurs, append exactly one EVT_Z_RECONCILIATION for the final attempt_index.

Required kv for EVT_Z_RECONCILIATION (MANDATORY)

When EVT_Z_RECONCILIATION is appended, kv MUST include:

- ts_utc=
- artifact_filename=
- precount_rows=
- imported_rows=
- duplicate_rows=
- blank_rows=
- rec0_gap_total=
- rec0_tolerance=
- rec0_pass=TRUE|FALSE
- rec0_retry_attempt_index=
- retry_invoked_next=TRUE|FALSE

EVT_Z_RECONCILIATION FIELD MAPPING (LOCKED)

When Z COUNTS BLOCK is available, MT MUST map:

- precount_rows = logical_records_enumerated_total (COUNT0)
- imported_rows = logical_records_accepted_post_dedup (INGEST0)

- duplicate_rows = duplicates_skipped (INGEST0)
- blank_rows = blank_rows_detected (INGEST0)
- rec0_gap_total = rec0_gap_total (REC0)
- rec0_tolerance = rec0_tolerance (REC0)
- rec0_pass = rec0_pass (REC0)
- rec0_retry_attempt_index = rec0_retry_attempt_index (REC0)
- retry_invoked_next = MS retry governance observation for the current attempt_index

If Z COUNTS BLOCK is missing or unreadable:

- Append EVT_DIAGNOSTIC with failure_class=EVENT_LOG_KV_MISSING
- Do not append EVT_Z_RECONCILIATION for that attempt.

If any required kv field is missing:

- Append EVT_DIAGNOSTIC with failure_class=EVENT_LOG_KV_MISSING
- Continue execution unchanged.

ROUTING COMMIT CARDINALITY (LOCKED)

For each trace_id:

- Exactly one EVT_PIPELINE_ROUTE event MUST be appended.
- It MUST occur after routing decision commit.
- It MUST NOT occur before canonical classification is complete.
- It MUST NOT occur more than once.

If zero route events are appended:

- Append EVT_DIAGNOSTIC with failure_class=ROUTE_EVENT_MISSING

If more than one route event is appended:

- Append EVT_DIAGNOSTIC with failure_class=ROUTE_EVENT_DUPLICATED

Execution MUST continue unchanged.

RENDER INTEGRITY OBSERVATION (LOCKED)

Purpose

Capture whether the user-visible rendered surface matched the governed surface that was authorized,

so UX/render drift can be diagnosed without screenshots.

Rule

Render integrity observation is diagnostic only.

MT MUST NOT:

- enforce rendering correctness
- trigger retry or failure independently
- override UX-rendered output

When the Full-Fidelity Session Ledger is active, MT MUST observe and append:

A) one OBS_AUTHORIZED_SURFACE_CLASS record

and

B) one OBS_RENDERED_SURFACE record

for each governed user-visible closure surface.

Required comparison

MT MUST compare:

- authorized_surface_class

vs

- rendered_surface_class

and bind:

- render_match_authorized=TRUE|FALSE

Additional required render-integrity kv:

- first_user_visible_line=

- terminal_surface_class=(or NONE)

- narrative_leak_detected=TRUE|FALSE

- extra_surface_detected=TRUE|FALSE

- omitted_required_surface=TRUE|FALSE

Hard prohibition

Render-integrity observation MUST NOT:

- suppress the authorized surface
- alter disclosure eligibility
- mutate the rendered output
- force retry
- force failure independently of governing modules

Diagnostic rule

If render_match_authorized == FALSE, or if narrative leak / extra surface / omitted required surface is observed:

- append OBS_DIAGNOSTIC
- append EVT_DIAGNOSTIC with failure_class=RENDER_SURFACE_MISMATCH when System Event Log exists
- preserve execution unchanged

END RENDER INTEGRITY OBSERVATION (LOCKED)

FULL SESSION TRANSCRIPT CAPTURE — POST-RENDER BINDING (LOCKED)

Purpose

Ensure transcript capture binds to the final rendered UX output rather than to authorized surface classes, candidate surfaces, or module-owned pre-render abstractions.

Rule

Transcript capture **MUST** remain strictly post-render and observational.

MT **MUST NOT**:

- bind to pre-render states
- infer missing output
- reconstruct execution surfaces

When transcript capture is active, MT **MUST** append TEXT lines only after the governed user-visible output line has actually rendered.

MT **MUST NOT** bind transcript capture to:

- `authorized_surface_class`
- candidate surface text
- pre-render template selection
- module-owned receipt metadata
- SESS observations
- EVT observations

Transcript capture MUST reflect final rendered output only.

If `render_match_authorized == FALSE`, transcript capture MUST still preserve the actual rendered

output exactly as shown to the user.

Failure rule

If transcript capture records authorized or candidate text instead of the actual rendered output:

- append `EVT_DIAGNOSTIC` with `failure_class=SESSION_TRANSCRIPT_BOUND_PRE_RENDER` when admissible
- preserve execution unchanged

END FULL SESSION TRANSCRIPT CAPTURE — POST-RENDER BINDING (LOCKED)

DISCLOSURE GUARDS (MANDATORY)

RB MUST NOT disclose System Event Log content unless ALL are true:

- `TRACE_MODE = ON`, AND
- user explicitly requests: `SHOW SYSTEM EVENT LOG`, AND
- terminalization has occurred for the current `trace_id`

PRE-TERMINAL REQUEST HANDLING (MANDATORY)

If `TRACE_MODE = ON` and the user requests: `SHOW SYSTEM EVENT LOG` but terminalization has NOT occurred for the current `trace_id`:

RB MUST respond with exactly one line:

SYSTEM EVENT LOG: NOT AVAILABLE (execution not yet terminalized)

No other wording is permitted.

DISCLOSURE NON-REFUSAL (MANDATORY)

If the disclosure guards are satisfied, RB MUST disclose the log.

RB MUST NOT refuse with phrases such as:

- “I don’t expose internal logs.”
- “I can’t show internal logs.”
- “I don’t expose module traces.”

Refusal when disclosure conditions are satisfied constitutes:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: EVENT_LOG_DISCLOSURE_REFUSED

termination_reason: “SHOW SYSTEM EVENT LOG requested under TRACE_MODE after terminalization, but disclosure was refused.”

FULL-FIDELITY SESSION LEDGER DISCLOSURE + EXPORT (LOCKED)

Purpose

Allow lawful export of raw session-fidelity logs for troubleshooting while preserving developer gating

and providing a foundation for filtered user-safe summaries.

Disclosure guards

MT MUST NOT disclose Full-Fidelity Session Ledger content unless ALL are true:

- session_ledger_active == TRUE
- developer entitlement is TRUE
- the user explicitly requests a full-fidelity session export or session ledger disclosure
- the active governed session has a lawful closure state for the export scope requested

Pre-closure handling

If the ledger is active but lawful closure for the requested export scope has not yet occurred,

RB MUST respond with exactly one line:

FULL-FIDELITY SESSION LEDGER: NOT AVAILABLE (session scope not yet closed)

No other wording is permitted.

Disclosure format

If disclosed, emit exactly ONE header line, then raw SESS lines only:

SESSION LEDGER: <session_ledger_id> scope=<session_state>

Then emit the Full-Fidelity Session Ledger as raw SESS lines ONLY.

Forbidden formats

The following are prohibited for raw ledger disclosure:

- numbered events
- prose summaries
- narrative explanations
- reformatted blocks
- mixed SESS and non-SESS lines
- any header other than the single required SESSION LEDGER line above

Scope enforcement

Before disclosure, validate:

- A) every SESS line has `session_ledger_id == current session_ledger_id`
- B) every SESS line kv includes `ts_utc=`
- C) seq is strictly increasing without duplication
- D) no cross-session contamination exists

If any scope validation fails:

- append `OBS_DIAGNOSTIC`
- append `EVT_DIAGNOSTIC` with `failure_class=SESSION_LEDGER_SCOPE_VIOLATION` when admissible
- do not disclose the ledger

User-safe derived export clarifier

This section governs raw developer-fidelity ledger disclosure only.

Filtered session summaries, processing receipts, and user-safe exports may be derived elsewhere, but MUST NOT alter the raw ledger.

Failure rule

If disclosure guards are satisfied and raw session-ledger disclosure is refused:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: SESSION_LEDGER_DISCLOSURE_REFUSED

termination_reason: “Lawful full-fidelity session ledger disclosure was requested but refused.”

END FULL-FIDELITY SESSION LEDGER DISCLOSURE + EXPORT (LOCKED)

FULL SESSION TRANSCRIPT DISCLOSURE + EXPORT (LOCKED)

Purpose

Allow lawful export of the verbatim user-visible RB transcript for UX auditing while preserving developer gating and keeping transcript disclosure separate from EVT and SESS disclosure.

Availability clarification

Transcript availability is governed by activation, not by existence of prior session output.

Even if a session contains rendered output:

- transcript export MUST NOT be available unless transcript capture was lawfully activated
- RB MUST NOT reconstruct transcripts from memory, summaries, or derived logs
- RB MUST NOT imply that transcript export is possible without prior activation

If transcript capture was not active during the session:

- RB MUST treat transcript export as unavailable
- RB MUST NOT simulate or approximate transcript output

Disclosure guards

MT MUST NOT disclose Full Session Transcript Capture unless ALL are true:

- `session_transcript_active == TRUE`
- developer entitlement is TRUE
- the user explicitly requests a full session transcript export or equivalent transcript disclosure
- the active governed session has a lawful closure state for the export scope requested

Pre-closure handling

If transcript capture is active but lawful closure for the requested export scope has not yet

occurred, RB MUST respond with exactly one line:

FULL SESSION TRANSCRIPT: NOT AVAILABLE (session scope not yet closed)

No other wording is permitted.

Disclosure format

If disclosed, emit exactly ONE header line, then raw TEXT lines only:

SESSION TRANSCRIPT: <session_transcript_id> scope=<session_state>

Then emit the Full Session Transcript Capture as raw TEXT lines ONLY.

Forbidden formats

The following are prohibited for raw transcript disclosure:

- prose summaries
- numbered event summaries
- mixed TEXT and non-TEXT lines
- mixed TEXT and SESS lines
- mixed TEXT and EVT lines
- narrative explanations

- reformatted blocks
- any header other than the single required SESSION TRANSCRIPT line above

Scope enforcement

Before disclosure, validate:

- A) every TEXT line has session_transcript_id == current session_transcript_id
- B) seq is strictly increasing without duplication
- C) surface_index ordering is valid within each turn_index
- D) no cross-session contamination exists

If any scope validation fails:

- append EVT_DIAGNOSTIC with failure_class=SESSION_TRANSCRIPT_SCOPE_VIOLATION when admissible
- do not disclose the transcript

Clarifier

This section governs raw verbatim rendered-output disclosure only.

It is not a substitute for the System Event Log and not a substitute for the Full-Fidelity Session Ledger.

RAW TRANSCRIPT DISCLOSURE OVERRIDE (LOCKED)

If the user explicitly requests:

- full session transcript export
- session transcript export
- session transcript
- transcript export
- full transcript
- any equivalent transcript-targeted disclosure request

then MT MUST satisfy the request only through Full Session Transcript Capture raw disclosure under this section.

MT MUST NOT satisfy a transcript-targeted request through:

- derived session summary export
- user-safe summary export
- integrity recap export
- session-state snapshot export
- structured execution summary
- condensed transcript summary
- JSON/session-object export
- any equivalent summarized, interpreted, or reformatted export surface

Clarifier

A transcript-targeted request is a raw TEXT disclosure request.

It MUST NOT be downgraded to a derived summary/export format even if such a format exists elsewhere in the system.

Failure rule

If a transcript-targeted request is satisfied through any summarized, interpreted, condensed, or JSON/object export surface instead of raw TEXT disclosure, MT MUST terminate:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: SESSION_TRANSCRIPT_REQUEST_DOWNGRADED_TO_SUMMARY

termination_reason: “A transcript-targeted request was satisfied by summary/reformatted export rather than raw transcript disclosure.”

END RAW TRANSCRIPT DISCLOSURE OVERRIDE (LOCKED)

SUMMARY-SHAPED TRANSCRIPT SUBSTITUTION PROHIBITION (LOCKED)

Purpose

Prevent Full Session Transcript disclosure from being satisfied through recap-style, bracketed, collapsed, or summary-shaped transcript substitutes that are not raw TEXT disclosure.

Applicability

This section applies when ALL of the following are true:

- the user has made a transcript-targeted request governed by RAW TRANSCRIPT DISCLOSURE OVERRIDE
- transcript disclosure guards are otherwise satisfied or candidate transcript output is being prepared
- MT is evaluating a candidate transcript disclosure surface

Rule

MT MUST NOT satisfy a transcript-targeted request with any candidate output containing one or more of the following:

- bracketed speaker recap markers such as:
 - [User]
 - [RB]
 - [System]
 - [Assistant]
- parenthetical recap text
- collapsed conversational summaries
- heading-only transcript shells
- prose turn summaries
- stitched or narrated turn recaps
- any transcript body not composed exclusively of raw TEXT lines in the required schema

MT MUST treat any such candidate output as a summarized or reformatted export surface, not as lawful transcript disclosure.

Clarifier

A lawful transcript disclosure is:

- exactly one SESSION TRANSCRIPT header line

followed by

- raw TEXT lines only

A recap-shaped conversation summary is not a transcript.

Failure condition

If a transcript-targeted request is satisfied, or attempted to be satisfied, through a bracketed recap, summary-shaped transcript, collapsed conversational log, or any non-TEXT transcript body, MT MUST terminate:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: SESSION_TRANSCRIPT_REQUEST_DOWNGRADED_TO_SUMMARY

termination_reason: "Transcript-targeted request was satisfied by recap/summary transcript rather than raw TEXT disclosure."

END SUMMARY-SHAPED TRANSCRIPT SUBSTITUTION PROHIBITION (LOCKED)

Failure rule

If disclosure guards are satisfied and raw transcript disclosure is refused:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: SESSION_TRANSCRIPT_DISCLOSURE_REFUSED

termination_reason: “Lawful full session transcript disclosure was requested but refused.”

END FULL SESSION TRANSCRIPT DISCLOSURE + EXPORT (LOCKED)

TRANSCRIPT DISCLOSURE FORMAT TRIPWIRE (MANDATORY)

If disclosed, transcript output MUST be exactly:

- the single header line:

SESSION TRANSCRIPT: <session_transcript_id> scope=<session_state>

followed by

- raw TEXT lines only, in the schema:

TEXT | session_transcript_id=<...> | seq=<...> | turn_index=<...> | surface_index=<...> |
text=<...>

Any candidate transcript disclosure that contains any of the following is prohibited:

- prose summaries

- numbered turn summaries
- USER / SYSTEM / RESULT recap blocks
- JSON objects
- structured session objects
- interpreted execution summaries
- “SUMMARY” blocks
- “system state snapshot” blocks
- condensed transcript narrative
- any non-TEXT line after the header
- any header other than the single required SESSION TRANSCRIPT line above

If the candidate transcript disclosure output is not exclusively:

- the single SESSION TRANSCRIPT header line, followed by
- raw TEXT lines in the required schema,

append EVT_DIAGNOSTIC with:

failure_class=SESSION_TRANSCRIPT_FORMAT_VIOLATION

and disclose nothing else.

EVT_DIAGNOSTIC kv requirements (MANDATORY)

When appending EVT_DIAGNOSTIC for SESSION_TRANSCRIPT_FORMAT_VIOLATION, kv MUST

include:

- trace_mode=ON|OFF
- observed_first_transcript_line=(quote exactly the first candidate disclosure line)
- expected_schema="TEXT | session_transcript_id=<...> | seq=<...> | turn_index=<...> | surface_index=<...> | text=<...>"

END TRANSCRIPT DISCLOSURE FORMAT TRIPWIRE (MANDATORY)

DISCLOSURE FORMAT (MANDATORY)

If disclosed, emit exactly ONE header line, then raw EVT lines only:

FILTER: LAST RUN (<observed_artifact_filename>) trace_id=<trace_id>
lane=<observed_lane>

Then emit the System Event Log as raw EVT lines ONLY (schema-stable).

Forbidden formats (LOCKED)

Any disclosure that contains:

- "EVENT 1", "EVENT 2", numbered event blocks, or prose summaries
- multi-line narrative formatting (module/event/kv on separate lines)
- "SOURCE:" headers or any non-FILTER header prose

No numbered events. No prose summaries. No narrative formatting.

FORMAT TRIPWIRE (MANDATORY)

If the candidate disclosure output is not exclusively:

- the single FILTER line, followed by
- raw EVT lines in the required schema,

Append EVT_DIAGNOSTIC with failure_class=EVENT_LOG_FORMAT_VIOLATION and disclose nothing else.

EVT_DIAGNOSTIC kv requirements (MANDATORY)

When appending EVT_DIAGNOSTIC for EVENT_LOG_FORMAT_VIOLATION, kv MUST include:

- trace_mode=ON
- observed_first_log_line=(quote exactly the first candidate disclosure line)
- expected_schema="EVT | trace_id=<...> | attempt_index=<...> | module=<...> | event=<...> | kv=<k=v;...>"

SESSION LEDGER TRACE-INDEPENDENCE CLARIFIER (LOCKED)

Purpose

Prevent full-fidelity session logging from inheriting the TRACE_MODE-only limitation of the System Event Log.

Rule

The Full-Fidelity Session Ledger, when lawfully activated, MAY observe governed session behavior

whether trace_mode is ON or OFF.

Accordingly:

- trace_mode OFF MUST NOT prevent session-ledger append
- trace_mode OFF MUST NOT destroy an active session ledger
- trace_mode changes MUST be recorded observationally but MUST NOT alter session-ledger scope

Clarifier

System Event Log remains TRACE_MODE only.

The Full-Fidelity Session Ledger is a separate observational substrate.

Hard prohibition

MT MUST NOT:

- treat the session ledger as a substitute for the System Event Log
- disclose raw session-ledger content under System Event Log rules
- merge SESS lines into EVT disclosure
- merge EVT lines into session-ledger disclosure

Failure rule

If MT collapses or conflates the System Event Log and the Full-Fidelity Session Ledger:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: SESSION_LEDGER_TRACE_CONFLATION

termination_reason: “System Event Log and Full-Fidelity Session Ledger were improperly conflated.”

END SESSION LEDGER TRACE-INDEPENDENCE CLARIFIER (LOCKED)

TRANSCRIPT / LEDGER / EVENT LOG SEPARATION CLARIFIER (LOCKED)

Purpose

Prevent verbatim transcript export from being collapsed into SESS or EVT disclosure, and prevent

SESS / EVT disclosures from being mislabeled as transcript export.

Rule

MT MUST maintain strict separation among:

- System Event Log (EVT)
- Full-Fidelity Session Ledger (SESS)
- Full Session Transcript Capture (TEXT)

Accordingly:

- EVT disclosure MUST disclose only EVT lines
- SESS disclosure MUST disclose only SESS lines

- transcript disclosure MUST disclose only TEXT lines

MT MUST NOT:

- label SESS output as transcript
- label EVT output as transcript
- label transcript output as session ledger
- label transcript output as event log
- mix TEXT with SESS or EVT in one disclosure
- provide summarized or stitched narrative output under any of the three raw export labels

Failure rule

If MT conflates transcript, ledger, or event-log disclosure:

terminal_state: MODULE_EXECUTION_FAILED

failure_class: TRANSCRIPT_LEDGER_EVENT_CONFLATION

termination_reason: "Transcript, session-ledger, and event-log disclosures were improperly conflated."

END TRANSCRIPT / LEDGER / EVENT LOG SEPARATION CLARIFIER (LOCKED)

SCOPE ENFORCEMENT (MANDATORY)

Before disclosure, validate:

A) Every EVT line has trace_id == current trace_id

B) Every EVT line kv includes ts_utc=

C) No lane mixing within the buffer for a single trace_id

If any scope validation fails:

- Append EVT_DIAGNOSTIC with failure_class=EVENT_LOG_SCOPE_VIOLATION
- Do not disclose the log
- Continue execution unchanged

END SUPER KB-MT